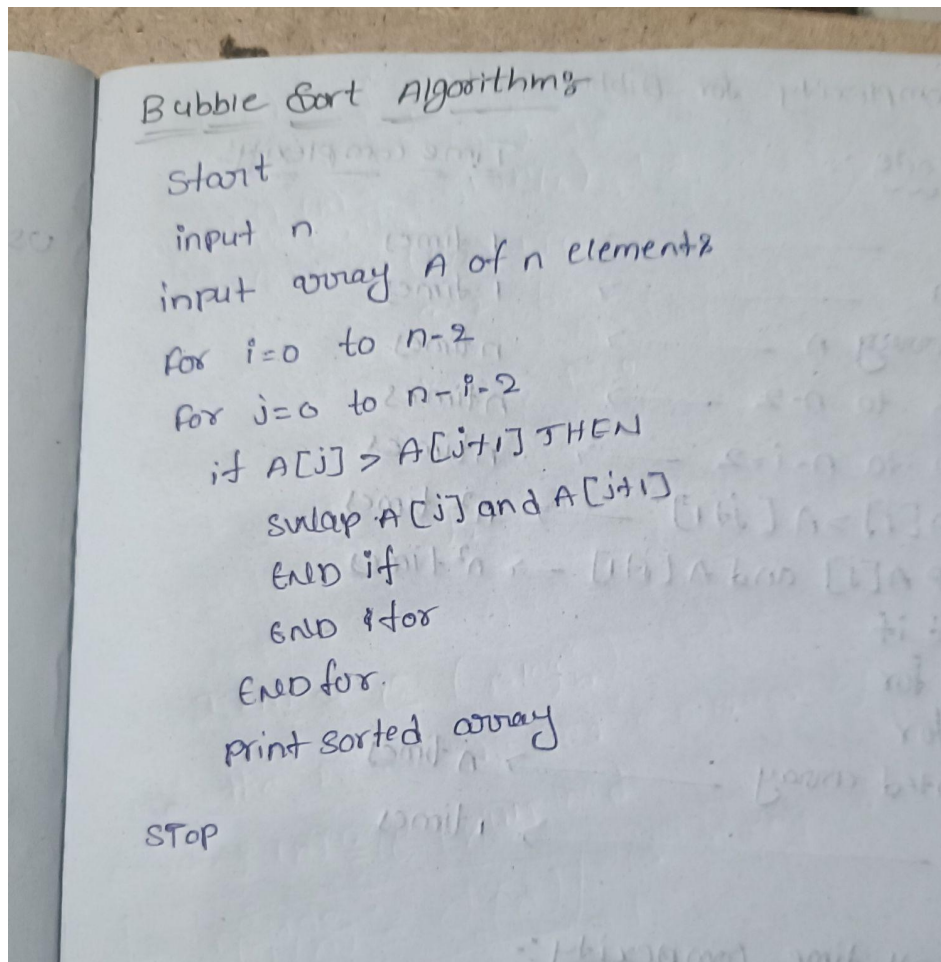


Practical 1

Aim:- To Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

Bubble sort

Algorithm :-



Code:-

```
#include <iostream> using
namespace std;
```

```
int main() { int n;
    cout << "Enter number of elements: "; cin >> n;
```

```

int arr[n];

cout << "Enter elements: "; for (int i
= 0; i < n; i++) {
    cin >> arr[i];
}

for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 1; j++) { if
(arr[j] > arr[j + 1]) {
        swap(arr[j], arr[j + 1]);
    }
    }
}

cout << "Sorted array: "; for (int
i = 0; i < n; i++) { cout << arr[i]
<< " ";
}

return 0;
}

```

Output:-

```

#include <iostream>

Enter number of elements: 5
Enter elements: 45 65 2 26 1
Sorted array: 1 2 26 45 65
Process returned 0 (0x0)    execution time : 31.877 s
Press any key to continue.

```

Time complexity:-

Time complexity for Bubble Sort:-

Pseudo code :-

Start $\rightarrow$ 1 times
input n $\rightarrow$ 1 times
input array A $\rightarrow$ n times
for i=0 to n-2 $\rightarrow$ n times
for j=0 to n-i-2 $\rightarrow$ n times
if $A[j] > A[j+1]$ $\rightarrow$ n^2 times
Swap $A[j]$ and $A[j+1]$ $\rightarrow$ n^2 times
End if
End for
End for
print sorted array $\rightarrow$ n times
Stop $\rightarrow$ 1 times

Time complexity:-

Now add all time complexity:-
 $T(n) = 1 + 1 + n + n + n + n^2 + n^2 + n + 1$
 $T(n) = 2n^2 + 4n + 3$

Wkt Time complexity properties "Avoid constant"
according that we get
 $T(n) = n^2 + n$

Taking maximum value from above equation.
 $T(n) = O(n^2)$

Final Time complexity:-

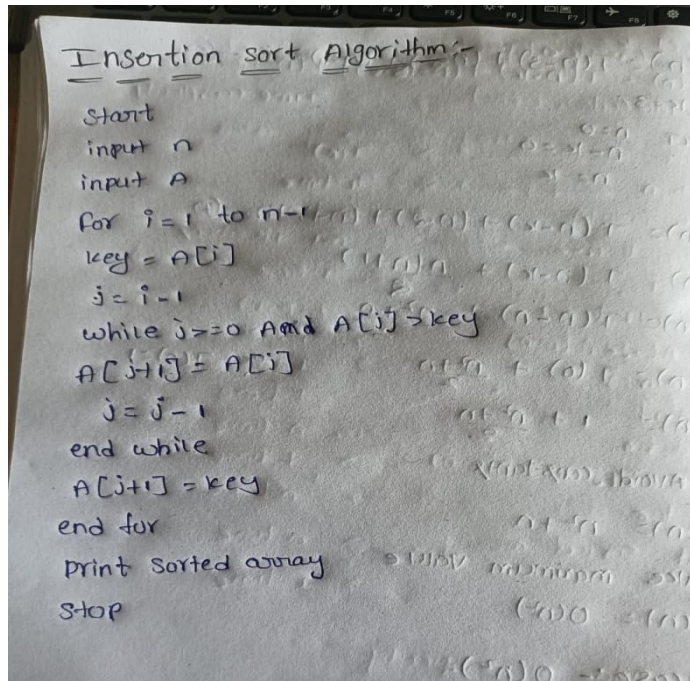
Best case:- $O(n^2)$
for example:- Array is already sorted
 $arr[] = 1\ 2\ 3\ 4\ 5$

Average case:- $O(n^2)$
for example:- Array is randomly arranged
 $arr[] = 3\ 1\ 5\ 2\ 4$

Worst case:-
for example:- Array is in reverse order
 $arr[] = 5\ 4\ 3\ 2\ 1$

Insertion sort

Algorithm :-



Code:-

```
#include <iostream> using
namespace std;

int main()
{
    int n, arr[100];

    cout << "Enter size of array: "; cin >> n;

    cout << "Enter array elements: "; for (int i
    = 0; i < n; i++)
    {
        cin >> arr[i];
```

```

    }

    // Insertion Sort
    for (int i = 1; i < n; i++)
    {
        int key = arr[i]; int j = i - 1;

        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j]; j--;
        }

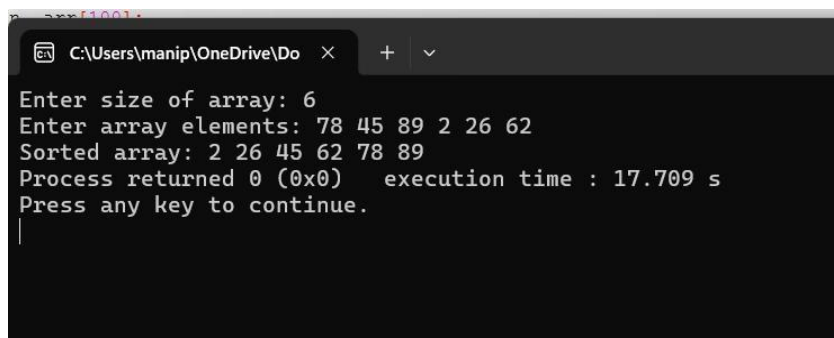
        arr[j + 1] = key;
    }

    cout << "Sorted array: "; for (int i = 0; i <
n; i++)
    {
        cout << arr[i] << " ";
    }

    return 0;
}

```

Output:-



```

C:\Users\manip\OneDrive\Do
Enter size of array: 6
Enter array elements: 78 45 89 2 26 62
Sorted array: 2 26 45 62 78 89
Process returned 0 (0x0)   execution time : 17.709 s
Press any key to continue.

```


Time complexity:-

Time complexity for insertion sort:-

By using Recurrence Relation:-

→ first sort $n-1$ element $\rightarrow T(n-1)$

→ in the worst case, insertion requires $n-1$ comparisons

$$T(n) = T(n-1) + n-1$$

using substitution method:-

$$T(n) = \begin{cases} T(n-1) + n-1 & n > 0 \\ T(0) = 0 & n = 0 \end{cases}$$

Now,

$$T(n) = T(n-1) + (n-1) \quad \text{--- (1)}$$

Let; $n = n-1$ put in eqn (1)

$$T(n-1) = T(n-2) + (n-2) \quad \text{--- (2)}$$

Substitute eqn (2) in eqn (1)

$$T(n) = T(n-2) + (n-2) + (n-1) \quad \text{--- (3)}$$

Let; $n = n-2$ put in eqn (1)

$$T(n-2) = T(n-3) + (n-3) \quad \text{--- (4)}$$

Substitute eqn (4) in eqn (3)

$$T(n) = T(n-3) + (n-3) + (n-2) + (n-1) \quad \text{--- (5)}$$

After some step

Wget:-

$$T(n) = T(n-3) + (n-3) + (n-2) + (n-1)$$

Here $n = 0$; $n = 0$

$$n = 14$$

$$T(n) = T(n-3) + (n-3) + (n-2) + (n-1)$$

$$T(n) = T(0) + \frac{n(n-1)}{2}$$

Wkt: $(n-3) + (n-2) + (n-1)$

$$= \frac{n(n-1)}{2}$$

$$T(n) = 1 + \frac{n^2 - n}{2}$$

Avoid constant

$$T(n) = n^2 - n$$

Taking maximum value in above

$$T(n) = O(n^2)$$

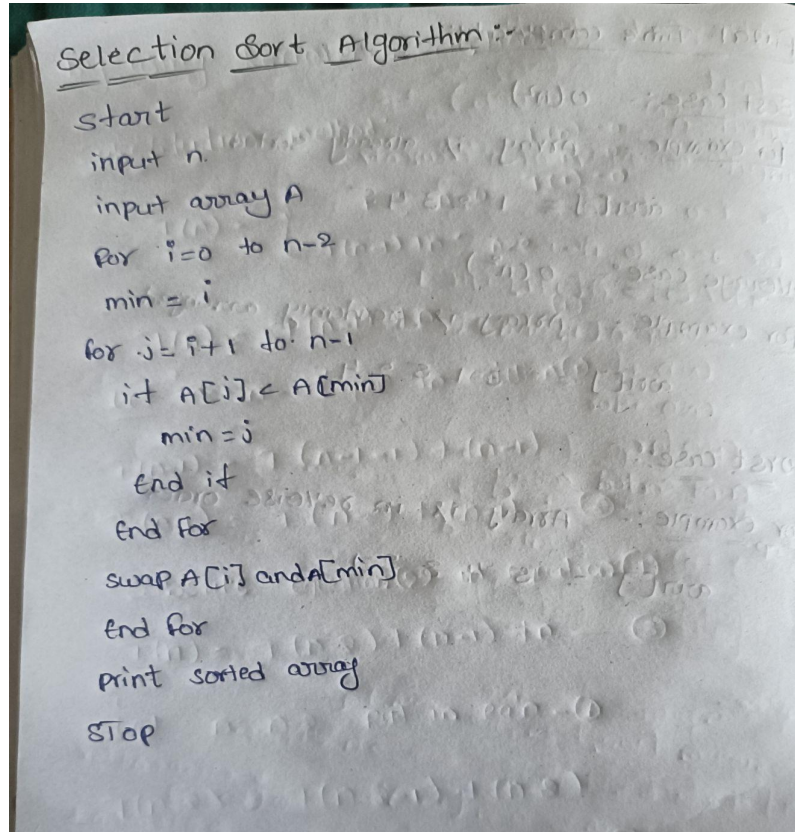
Best case:- $O(n)$ → array is already sorted

Average:- $O(n^2)$ → array is randomly arranged

Worst case:- $O(n^2)$ → array is in reverse order

Selection sort

Algorithm :-

A photograph of a piece of paper with the selection sort algorithm written in cursive. The text is as follows:

Selection Sort Algorithm
start
input n
input array A
for i=0 to n-2
 min = i
 for j=i+1 to n-1
 if A[j] < A[min]
 min = j
 end if
 end for
 swap A[i] and A[min]
end for
print sorted array
stop

Code:-

```
#include <iostream> using
namespace std;
int main()
{
    int n, temp;

    cout << "Enter size: "; cin >>
    n;

    int A[n];
```

```
cout << "Enter elements: "; for (int i
= 0; i < n; i++)
{
    cin >> A[i];
}

// Selection Sort
for (int i = 0; i < n - 1; i++)
{
    int min = i;

    for (int j = i + 1; j < n; j++)
    {
        if (A[j] < A[min])
        {
            min = j;
        }
    }

    // Swap temp =
    A[i];
    A[i] = A[min]; A[min] =
    temp;
}

cout << "Sorted array: "; for (int i

= 0; i < n; i++)
{
    cout << A[i] << " ";
}

return 0;
}
```


Output:-

```
cpp - Code::Blocks 20.03
C:\Users\manip\OneDrive\Do x + v
Enter size: 5
Enter elements: 8 65 26 334 1
Sorted array: 1 8 26 65 334
Process returned 0 (0x0)   execution time : 22.201 s
Press any key to continue.
```

Time complexity:-

Time complexity for selection sort :-

Pseudo code:-

start $\rightarrow T(n)$

input n $\rightarrow$ 1 time

input array A $\rightarrow$ n time

for i=0 to n-2 $\rightarrow$ n-1 iterations

min = i $\rightarrow$ 1 time

for j=i+1 to n-1 $\rightarrow$ $T(n-1)$

if A[j] < A[min] $\rightarrow$ 1 time

min = j $\rightarrow$ 1 time

end if $\rightarrow$ 1 time

end for $\rightarrow$ n time

swap A[j] and A[min] $\rightarrow$ 1 time

end for $\rightarrow$ n time

print sorted array $\rightarrow$ n time

stop $\rightarrow$ 1 time

Now add all complexity

$$T(n) = T(n-1) + n + n + n + n - 1 + 6$$
$$T(n) = T(n-1) + 5n + 5$$

•• Avoid constants

$$T(n) = T(n-1) + n$$

By using Recurrence Relation

Substitution method:-

$$T(n) = \begin{cases} 1, & n=0 \\ T(n-1) + n, & n>0 \end{cases}$$
$$T(n) = T(n-1) + n \quad \text{--- (1)}$$

Let $n = n-1$ put in equation (1)

$$T(n-1) = T(n-2) + (n-1)$$
$$T(n-1) = T(n-2) + (n-1) \quad \text{--- (2)}$$

Substitute equation (2) in equation (1)

$$T(n) = T(n-2) + (n-1) + n \quad \text{--- (3)}$$

Let $n = n-2$ put in eqn (1)

$$T(n-2) = T(n-3) + (n-2)$$
$$T(n-2) = T(n-3) + (n-2) \quad \text{--- (4)}$$

Substitute eqn (4) in eqn (3)

$$T(n) = T(n-3) + (n-2) + (n-1) + n \quad \text{--- (5)}$$

After same steps we get

$$T(n) = T(n-3) + (n-3) + (n-2) + (n-1) + n \quad \text{--- (6)}$$

Let $k=3$

WKT $n=0$
 $n-k=0$
 $n=k$

$$T(n) = T(n-k) + (n-k) + (n-1) + n \quad \text{--- (6)}$$
$$T(n) = T(n-k) + \frac{n(n+1)}{2}$$
$$T(n) = T(n-n) + \frac{n^2+n}{2}$$

$\because T(0)=1$

$$T(n) = 1 + \frac{n^2+n}{2}$$

Avoid constants

$$T(n) = n^2 + n$$

Take maximum value

$$T(n) = O(n^2)$$

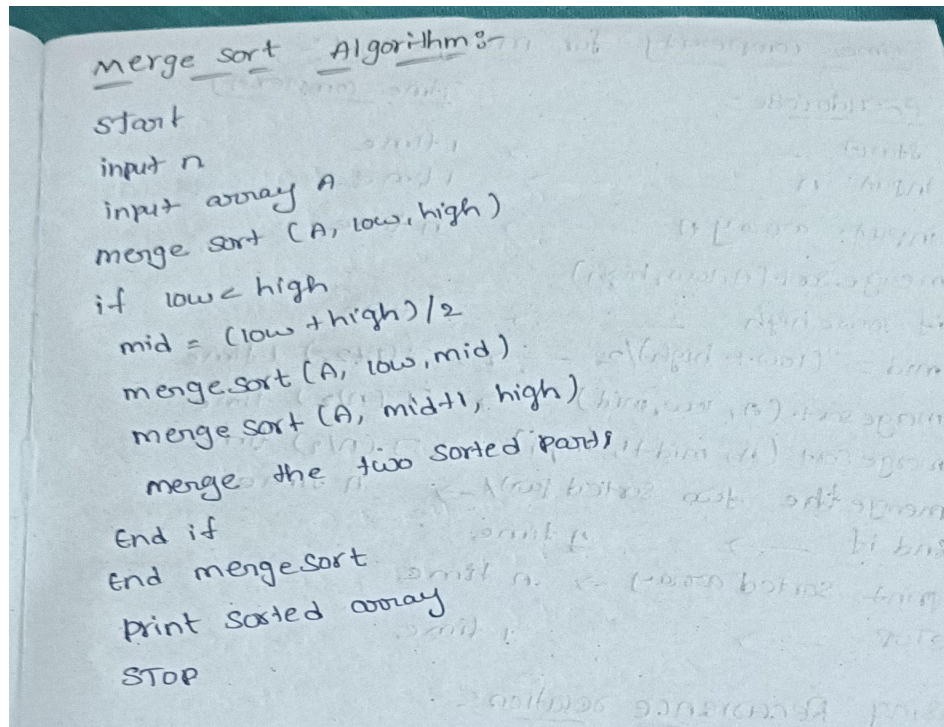
Best case:- $O(n^2)$
Array is already sorted

Average case:- $O(n^2)$
Array is randomly arranged

Worst case:-
Array is in reverse order

Merge Sort

Algorithm :-



Code:-

```
#include <iostream>

using namespace std;

void merge(int arr[], int low, int mid, int high)
{
    int i = low;
    int j = mid + 1;
    int k = 0;

    int temp[100];

    while (i <= mid && j <= high)
    {
        if (arr[i] < arr[j])
        {

```

```
        temp[k] = arr[i];
        i++;
    }
    else
    {
        temp[k] = arr[j];
        j++;
    }
    k++;
}
```

```
while (i <= mid)
{
    temp[k] = arr[i];
    i++;
    k++;
}
```

```
while (j <= high)
{
    temp[k] = arr[j];
    j++;
    k++;
}
```

```
for (i = low, k = 0; i <= high; i++, k++)
{
    arr[i] = temp[k];
}
}
```

```
void mergeSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int mid = (low + high) / 2;
```

```
mergeSort(arr, low, mid);  
mergeSort(arr, mid + 1, high);
```

```
merge(arr, low, mid, high);  
}  
}
```

```
int main()  
{  
    int n, arr[100];  
  
    cout << "Enter size: ";  
    cin >> n;  
  
    cout << "Enter elements: ";  
    for (int i = 0; i < n; i++)  
    {  
        cin >> arr[i];  
    }  
  
    mergeSort(arr, 0, n - 1);  
  
    cout << "Sorted array: ";  
    for (int i = 0; i < n; i++)  
    {  
        cout << arr[i] << " ";  
    }  
  
    return 0;  
}
```

Output:-

```
for (i = low, k = 0; i <= high; i++, k++)  
C:\Users\manip\OneDrive\Do X + v  
Enter size: 8  
Enter elements: 67 34 99 2 65 44 126  
26  
Sorted array: 2 26 34 44 65 67 99 126  
Process returned 0 (0x0) execution time : 38.211 s  
Press any key to continue.
```

Time complexity:-

Time complexity for merge sort:-

Pseudo Code:-

start $\rightarrow$ 1 time
Input n $\rightarrow$ 1 time
input array A $\rightarrow$ n time
mergeSort(A, low, high) $\rightarrow$ 1 time
if low < high $\rightarrow$ 1 time
mid = (low + high) / 2 $\rightarrow$ 1 time
mergeSort(A, low, mid) $\rightarrow$ T(n/2) time
mergeSort(A, mid+1, high) $\rightarrow$ T(n/2) time
merge the two sorted parts $\rightarrow$ n time
end if $\rightarrow$ 1 time
print sorted array $\rightarrow$ n time
stop $\rightarrow$ 1 time

using Recurrence relation:-

$$T(n) = 2T(n/2) + O(n)$$

By using master method:-

General equation = $T(n) = aT(n/b) + f(n)$

$$f(n) = O(n^k * (\log^p n))$$

Here $a = 2$ $b = 2$
 $k = 1$ $p = 0$

Here = $\log_b a = \log_2 2$
 $k = 1$

So using case 2, Here $p = 0$,
Now $p > -1$ we get $O(n^k * \log^{p+1} n)$
 $T(n) = O(n^1 * \log^{0+1} n)$ $\therefore k=1$
 $T(n) = O(n * \log n)$
 $T(n) = O(n \log n)$

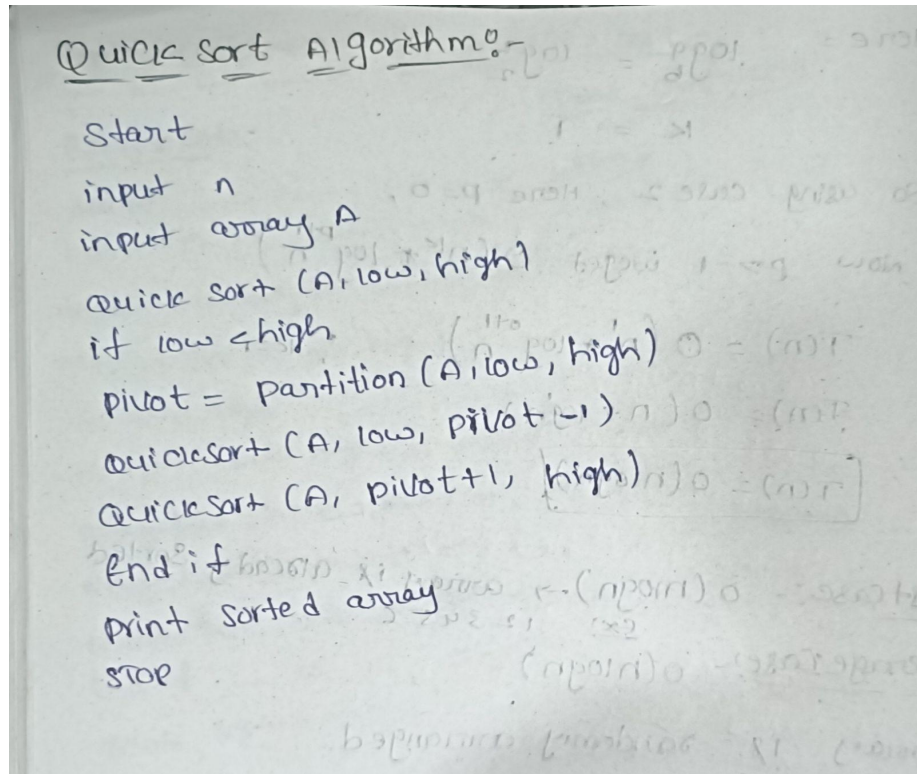
Best case:- $O(n \log n) \rightarrow$ array is already sorted
Ex: 1 2 3 4 5 6

Average case:- $O(n \log n)$
array is randomly arranged
eg: arr[] = 3 5 6 7 8 1

worst case:- $O(n \log n)$
array is in reverse order
eg:- arr[] = 5 4 3 2 1

Quick sort

Algorithm :-



Handwritten algorithm for Quick Sort:

```
Quick sort Algorithm:-  
Start  
input n  
input array A  
quick sort (A, low, high)  
if low < high  
    pivot = partition (A, low, high)  
    quicksort (A, low, pivot-1)  
    quicksort (A, pivot+1, high)  
end if  
Print sorted array  
stop
```

Code:-

```
#include <iostream>  
using namespace std;  
  
int partition(int arr[], int low, int high)  
{  
    int pivot = arr[high];  
    int i = low - 1;  
  
    for (int j = low; j < high; j++)  
    {  
        if (arr[j] < pivot)  
        {  
            i++;  
            swap(arr[i], arr[j]);  
        }  
    }  
}
```



```

        swap(arr[i + 1], arr[high]);

        return i + 1;
    }

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pivot = partition(arr, low, high);

        // 1st recursive call → T(n/2)
        quickSort(arr, low, pivot - 1);

        // 2nd recursive call → T(n/2)
        quickSort(arr, pivot + 1, high);
    }
}

int main()
{
    int n;
    int arr[100];

    cout << "Enter size: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }

    quickSort(arr, 0, n - 1);
}

```

```

cout << "Sorted array: ";
for (int i = 0; i < n; i++)
{
    cout << arr[i] << " ";
}

return 0;
}

```

Output:-

```

C:\Users\manip\OneDrive\Do x + v
Enter size: 5
Enter elements: 67 54 87 26 62
Sorted array: 26 54 62 67 87
Process returned 0 (0x0)   execution time : 16.901 s
Press any key to continue.

```

Time complexity:-

Time complexity for merge sort:-

Pseudo Code:-

Start $\rightarrow$ 1 time
 input n $\rightarrow$ 1 time
 input array A $\rightarrow$ n time
 quicksort(A, low, high) $\rightarrow$ 1 time
 if low < high $\rightarrow$ 1 time
 pivot = partition(A, low, high) $\rightarrow$ ~~1 time~~ time $O(n)$
 quicksort(A, low, pivot-1) $\rightarrow$ $T(n/2)$ time
 quicksort(A, pivot+1, high) $\rightarrow$ $T(n/2)$ time
 end if $\rightarrow$ 1 time
 print sorted array $\rightarrow$ n time
 stop $\rightarrow$ 1 time

using Recurrence relation:-
 when the pivot divides the array, we get
 $T(n) = 2T(n/2) + O(n)$

By using master method
 General equation:-
 $T(n) = aT(n/b) + f(n)$
 Here, $f(n) = O(n^1 \times \log^p n)$
 Here, $a=2$, $b=2$
 $k=0$, $p=0$

Now $\log_b a = \log_2 2$
 $\log_2 2 = 1$
 $k=1$

By using case $k=1$, Here $p=0$
 So $T(n) = O(n^1 \log^1 n)$
 $T(n) = O(n \log n)$

Best case:- $O(n \log n)$
 array is already sorted
 eg:- 1 2 3 4 5

Average case:- $O(n \log n)$
 array is randomly arranged
 eg:- 3 5 6 7 1 2 5

Worst case:-
 array is in reverse order
 eg:- 4 5 3 5 2 5 1 5 5

Conclusion:-

The implementation and time analysis of **Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort** helped us understand how different sorting algorithms work and how efficiently they sort data. Bubble, Selection, and Insertion Sort are simple and easy to implement but generally take **$O(n^2)$** time. Merge Sort provides consistent **$O(n \log n)$** performance, while Quick Sort gives **$O(n \log n)$** average performance but can take **$O(n^2)$** in the worst case. Therefore, the choice of sorting algorithm depends on the size of the data and the required performance